

✓ PDSP 2025, Lecture 05, 21 August 2025

✓ Conditional statement

`if` allows conditional execution

```
if condition:
    statement 1
    ...
    statement k
else:
    statement 1'
    ...
    statement k'
```

- If `condition` evaluates to `True`, the first block is executed, otherwise the second block.
- The `else:` block is optional. If there is no `else:` block and the `condition` evaluates to `False`, execution skips over to the next statement after the `if`
- Example: Compute the absolute value of a number

```
def myabs(x): # myabs to avoid any confusion with built-in abs()
    if x < 0:
        return(-x)
    else:
        return(x)
```

```
myabs(-9), myabs(7)
```

```
(9, 7)
```

✓ Multiway branching --- `elif`

Suppose we want to compute $sign(x) = \begin{cases} x < 0 & = -1, \\ x = 0 & = 0, \\ x > 0 & = 1 \end{cases}$

In Python, we would have to nest `if` statements like this:

```
if x < 0:
    return(-1)
else:
    if x == 0:
        return(0):
    else:
        return(1)
```

- As we see, the indentation of the nested `if` pushes the code to the right
- With more cases, this would become worse
- Python provides `elif` to avoid this cascaded nesting

```
if x < 0:
    return(-1)
elif x == 0:
    return(0):
else:
    return(1)
```

- Can have as many `elif` blocks as you need
- `else` is still optional

```
def sign(x):
    if x < 0:
        return(-1)
```

```

elif x == 0:
    return(0)
else:
    return(1)

```

```
sign(-7)
```

```
-1
```

```
sign(8)
```

```
1
```

```
sign(0)
```

```
0
```

Lists

- Sequences of values, indexed by position
- For a list with `n` values, valid positions are `0` to `n-1`
 - `len(l)` gives the length of a list
- Accessing a position beyond `len(l)-1` results in `IndexError`

```
l = list(range(20,40))
```

```
len(l), l[3], l[19]
```

```
(20, 23, 39)
```

```
l[20]
```

```

-----
IndexError                                Traceback (most recent call last)
Cell In[9], line 1
----> 1 l[20]

IndexError: list index out of range

```

- What about indices below `0`?
- Index `-j` is interpreted as `len(l)-j`
 - Useful for accessing values from the end of the list
 - Valid indices in reverse are `-1`, `-2`, ..., `-len(l)`

```
l[-1], l[-20]
```

```
(39, 20)
```

Slices

- Recall that `nprimes(n)` computed the first `n` primes

```

def isprime(n):
    for j in range(2,n):
        if n % j == 0:
            return(False)
    return(True)

def nprimes(n):
    plist = []
    j = 2
    while (len(plist) < n):
        if isprime(j):
            plist.append(j)
        j = j+1
    return(plist)

```

```
first20primes = nprimes(20)
```

```
first20primes
```

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71]
```

- What are the primes from 11 to 15?
- Need a sublist of the original list
- `l[i:j]` is the list `[l[i], l[i+1], ..., l[j-1]]`
- Similar to

```
newl = []
for k in range(i,j):
    newl.append(k)
```

```
first20primes[11:16]
```

```
[37, 41, 43, 47, 53]
```

- like `range()` if the indices don't make sense, you get an empty list

```
first20primes[11:10]
```

```
[]
```

- Unlike accessing `l[i]`, can give upper bound beyond the list
- `l[i:len(l)+10]` is interpreted as `l[i:len(l)]`

```
first20primes[11:40]
```

```
[37, 41, 43, 47, 53, 59, 61, 67, 71]
```

- Can omit the upper bound, defaults to `len(l)`

```
first20primes[15:]
```

```
[53, 59, 61, 67, 71]
```

- Likewise, omit the lower bound, defaults to `0`

```
first20primes[:10]
```

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

- Omit both lower and upper bound to get a `full slice`
 - Full slice returns a new list that is a copy of the list
 - Significance will become clearer later

```
first20primes[:]
```

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71]
```

▼ More about `range()`

- `range(i, j)` generates the sequence `i, i+1, ..., j-1`
- `range(n)` generates the sequence `0, 1, ..., n-1` -- implicitly starts with `0`
- What if we want to skip over some numbers
 - All even numbers from `4` to `40`
- Optional third argument is the step size
 - `range(i, j, k)` is `i, i+k, ..., i+mk` for the largest `m` such that `i+mk < j` and `i+(m+1)k >= j`

```
list(range(4,41,2)) # Even numbers from 4 to 40
```

```
[4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40]
```

```
list(range(4,40,2)) # Even numbers from 4 to 38
```

```
[4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38]
```

- Can also count down -- give a negative step!

```
list(range(10,0,-1))
```

```
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

- `range(i,j,k)` generate `i, i+k, ...` so that the sequence does not *cross* `j`
- Depending on whether it is increasing or decreasing, the last value will be less than `j` or greater than `j`

✓ Stepped slices

- Can similarly give a third argument in a slice

```
first20primes[2:20:3]
```

```
[5, 13, 23, 37, 47, 61]
```

```
first20primes[19:0:-1]
```

```
[71, 67, 61, 59, 53, 47, 43, 41, 37, 31, 29, 23, 19, 17, 13, 11, 7, 5, 3]
```

- Explain the following output. (Hint, what is `l[-1]`?)

```
first20primes[19:-1:-1]
```

```
[]
```

- Can omit upper and lower bounds but give a step
- `l[::-1]` is the entire list in reverse
 - Note that the default lower and upper bound are determined by the step

```
first20primes[::-1]
```

```
[71, 67, 61, 59, 53, 47, 43, 41, 37, 31, 29, 23, 19, 17, 13, 11, 7, 5, 3, 2]
```

✓ Assigning slices

- Can assign a list to a slice

```
l = list(range(20,30))
```

```
l[3:6] = [53,54,55]
```

```
l
```

```
[20, 21, 22, 53, 54, 55, 26, 27, 28, 29]
```

- Can contract or expand the slice when reassigning
- Indices of values to the right will change

```
l = list(range(20,30))
```

```
l[3:5] = [53,54,55,63,64,65]
```

```
l
```

```
[20, 21, 22, 53, 54, 55, 63, 64, 65, 25, 26, 27, 28, 29]
```

```
l = list(range(20,30))
```

```
l[3:5] = []
```

```
l
```

```
[20, 21, 22, 25, 26, 27, 28, 29]
```

Operations on lists

- Recall that `+` concatenates two lists
- Returns a new list
- Original lists are unchanged

```
l1 = [1,2,3]
l2 = [4,5,6]
l3 = l1 + l2
```

```
l3, l1, l2
```

```
([1, 2, 3, 4, 5, 6], [1, 2, 3], [4, 5, 6])
```

- A useful invariant about slices
- For any list `l`, and any integer `j`, `l == l[:j] + l[j:]`

```
l3[:-1]+l3[-1:]
```

```
[1, 2, 3, 4, 5, 6]
```

```
l3[:2]+l3[2:]
```

```
[1, 2, 3, 4, 5, 6]
```

```
l3[:9]+l3[9:]
```

```
[1, 2, 3, 4, 5, 6]
```

Applying functions to lists

- `l.append(v)` is the same as `l = l+[v]`
 - Ask the list `l` to append `v` to itself
 - `l.append(v)` updates `l` in place
 - `l = l+[v]` creates a new list and reassigns the list pointed to by `l`
 - Again, we will see the significance of this later

```
l3.append(7)
```

```
l3
```

```
[1, 2, 3, 4, 5, 6, 7]
```

- It is a mistake to reassign a list after an `append()`

```
l3 = l3.append(8)
```

```
l3
```

- Assignment `v = e` stores return value of `e` in `v`
- Return value of `l.append()` is empty

Other functions

- `l.insert(pos, val)` inserts `val` at position `p`
 - Similar to `l = l[:pos] + [val] + l[pos:]`
- `l.extend(newl)` extends `l` with a list of values `newl`
 - Similar to `l = l + newl`
- Like `l.append(v)`, these update the list in place, do not reassign return value

```
l3 = [1,2,3,4,5,6,7]
```

```
l3.insert(0,0)
```

```
l3
```

```
[0, 1, 2, 3, 4, 5, 6, 7]
```

```
l3.extend([8,9,10])
```

```
l3
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

▼ Sorting

- `l.sort()` sorts a list in place
 - Python allows lists of mixed types
 - To sort a list, the values must be of a uniform comparable type

```
l = [1,15,3,7,9,2]
```

```
l.sort()
```

```
l
```

```
[1, 2, 3, 7, 9, 15]
```

```
badl = [1,'CSK',True,7.5]
```

```
badl.sort()
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[52], line 1
----> 1 badl.sort()

TypeError: '<' not supported between instances of 'str' and 'int'
```

```
blist = [True,False]
```

```
blist.sort()
```

```
blist
```

```
[False, True]
```

- If you want a sorted copy of `l` without disturbing `l`, use `sorted(l)`

```
l = [15, 1000, 9, 7, 3, 2, 1]
```

```
l
```

```
[15, 1000, 9, 7, 3, 2, 1]
```

```
sorted(l)
```

```
[1, 2, 3, 7, 9, 15, 1000]
```

```
l
```

```
[15, 1000, 9, 7, 3, 2, 1]
```

- Can store a copy of the sorted list in another list

```
newl = sorted(l)
```

```
newl, l
```

```
([1, 2, 3, 7, 9, 15, 1000], [15, 1000, 9, 7, 3, 2, 1])
```

- Can we, instead, first copy `l` and sort the copy in place using `sort()`?

```
newl = l
```

```
newl.sort()
```

```
newl
```

```
[1, 2, 3, 7, 9, 15, 1000]
```

```
l
```

```
[1, 2, 3, 7, 9, 15, 1000]
```

- Sorting `newl` also sorts `l`
- This does not happen with types like `int`
- We will investigate this later

```
y = 7
```

```
x = y
```

```
x, y
```

```
(7, 7)
```

```
x = 17
```

```
x, y
```

```
(17, 7)
```

- Many other built-in functions on lists
 - `l.reverse()` reverses a list
- Look up Python documentation